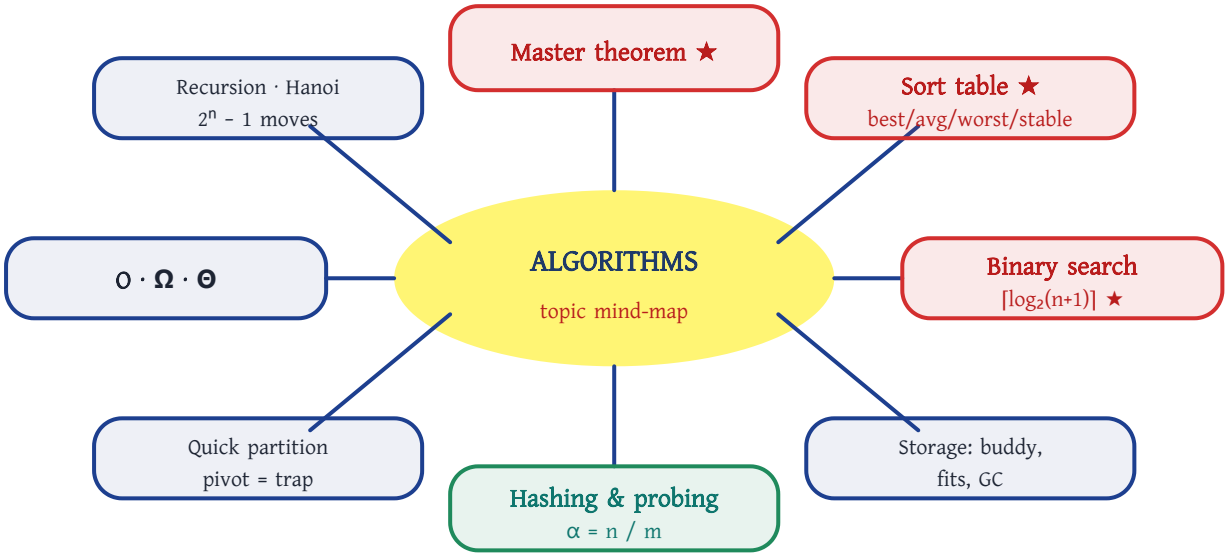
Recursion, Sorting, Searching & Hashing

Master theorem · sort table · probing · storage · exam-ready in 11 sheets



↑ the lesson on one page: red = high-yield, green = the numerical

Syllabus coverage

The syllabus line is "**Recursion, sorting, searching, hashing, storage management. Techniques for analysis of algorithms.**" Heap internals are in the Heaps & B-Trees lesson, divide-and-conquer / greedy / DP as design methods in the Design Methods lesson, and the lower-bound proof in the NP-Completeness lesson.

High-yield list

① Hanoi: $2^n - 1$ moves
② $2T(n/2) + n = \Theta(n \log n)$
③ $O \neq$ worst, $\Theta \neq$ average
④ Quick worst $O(n^2)$: bad pivot
⑤ Stable: merge, insertion, bubble
⑥ Not stable: quick, heap, selection
⑦ Binary search $\leq \lceil \log_2(n+1) \rceil$
⑧ Comparison sort $\geq \Omega(n \log n)$
⑨ Probes $\approx 1/(1 - \alpha)$
⑩ Ref-counting fails on cycles

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

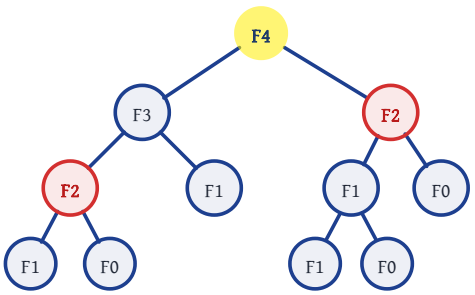
Contents

✓ Recursion & the call stack … p. 2
✓ Asymptotic notation & analysis … p. 3
✓ Recurrences & Master theorem … p. 4
✓ Simple, heap & linear-time sorts … p. 5
✓ Merge, quick & the sort table … p. 6
✓ Searching … p. 7
✓ Hashing & collisions … p. 8
✓ Storage management & GC … p. 9
✓ Night-before p. 10 · Practice p. 11

1 Recursion & the Call Stack

Definition: A **recursive** function solves a problem by calling itself on a **smaller** instance. It needs a **base case** that stops without a call and a **recursive case** that moves strictly towards it. Every call pushes an **activation record** (parameters, locals, return address) on the stack, so recursion depth d costs **O(d) space**.

Problem	Recurrence	Cost
Factorial	$n! = n \cdot (n-1)!$, $0! = 1$	$\Theta(n)$ time & stack
Fibonacci, naive	$F(n) = F(n-1) + F(n-2)$; $F(0)=0$, $F(1)=1$	**$\Theta(\varphi^n)$**, $\varphi \approx 1.618$ (quoted $O(2^n)$)
Fibonacci, memo / loop	each F(i) once	$\Theta(n)$
Tower of Hanoi	$M(n) = 2M(n-1) + 1$, $M(1) = 1$	**$2^n - 1$** moves
Fast power x^n	$x^n = (x^{\lfloor n/2 \rfloor})^2$ (× x if n odd)	$\Theta(\log n)$



F2 solved twice → 9 calls for F4

recursion tree of naive fib(4): overlapping sub-problems (red) explode exponentially

```c
/* not tail-recursive: × after return */
int fact(int n) {
  if (n == 0) return 1;      /* base */
  return n * fact(n - 1);
}
/* tail-recursive: call is the last act */
int factT(int n, int acc) {
  if (n == 0) return acc;
  return factT(n - 1, n * acc);
}
```

Kinds of recursion: **Direct**: f calls f. **Indirect**: f → g → f. **Linear**: one call per activation (factorial). **Tree**: two or more (Fibonacci, Hanoi). **Tail**: the call is the final action, so a compiler can reuse the frame (**tail-call elimination**) → O(1) stack. Every recursion can be rewritten as a loop with an explicit stack.

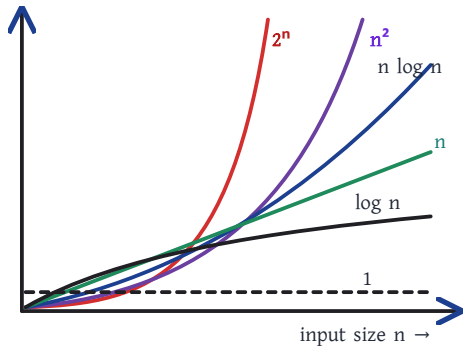
Worked example ✍ Tower of Hanoi: Move n − 1 discs A → B (via C), the largest A → C, then n − 1 discs B → C. So $M(n) = 2M(n-1) + 1 \Rightarrow M(n) + 1 = 2[M(n-1) + 1] \Rightarrow M(n) + 1 = 2^n \Rightarrow$ **$M(n) = 2^n - 1$**. n = 3: A→C, A→B, C→B, **A→C**, B→A, B→C, A→C = **7** moves; n = 4 → 15; n = 64 → $2^{64} - 1 \approx 1.8 \times 10^{19}$. The smallest disc moves on every odd step.

Memory trick: "**Base first, smaller next.**" Write the stop condition before the self-call, and check that the argument shrinks. Naive fib(n) makes **2F(n+1) − 1** calls: fib(4) → 9, fib(5) → 15.

Exam trap: A missing base case gives **stack overflow**, not an endless loop. Recursion is usually **slower** than the equivalent loop (call overhead). C and Java do **not** guarantee tail-call elimination. Recursion depth of naive fib(n) is only n, even though calls are exponential.

2 Asymptotic Notation & Analysis Techniques

Notation	Formal definition ($\exists\ c, n_0 > 0, \forall\ n \geq n_0$)	Meaning	Like
$f = O(g)$	$0 \leq f(n) \leq c \cdot g(n)$	asymptotic **upper bound**	$\leq$
$f = \Omega(g)$	$f(n) \geq c \cdot g(n) \geq 0$	asymptotic **lower bound**	$\geq$
$f = \Theta(g)$	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$	**tight bound**: both O and Ω	$=$
$f = o(g)$	for **every** $c > 0$; $\lim f/g = 0$	strictly slower growth	$<$
$f = \omega(g)$	$\lim f/g = \infty$	strictly faster growth	$>$



for large n the order never changes, whatever the constants

Growth ladder: $1 < \log\log n < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$. Log base does not matter ($\log_a n = \Theta(\log_b n)$), but exponent base does: $2^n = o(3^n)$. $\log(n!) = \Theta(n \log n)$; $1 + 2 + \ldots + n = \Theta(n^2)$; $1 + \frac{1}{2} + \ldots + 1/n \approx \ln n$.

Worked example ✍ prove a Θ bound: $f(n) = 3n^2 + 5n + 2$. For $n \geq 1$: $3n^2 \leq f(n) \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$. So $c_1 = 3$, $c_2 = 10$, $n_0 = 1$ $\Rightarrow$ **$f = \Theta(n^2)$**. Also $2^{n+1} = O(2^n)$ ($c = 2$) but $2^{2n} = 4^n \neq O(2^n)$.

Loop pattern	Basic steps	Order
for i = 1..n: for j = 1..i	$n(n+1)/2$	$\Theta(n^2)$
for (i = 1; i < n; i *= 2)	$\lceil \log_2 n \rceil$	$\Theta(\log n)$
for i = 1..n: for (j = i; j ≤ n; j += i)	$n(1 + \frac{1}{2} + \ldots + 1/n)$	$\Theta(n \log n)$
while (n > 2) n = √n	$\log_2 \log_2 n$	$\Theta(\log\log n)$

Common misconception: **O ≠ worst case**, Ω ≠ best case, Θ ≠ average case. Notations bound a **function**; cases pick the **input**. Insertion sort: worst case $\Theta(n^2)$, best case $\Theta(n)$; so its running time is $O(n^2)$ and $\Omega(n)$, but not $\Theta(n^2)$ for every input.

Best · average · worst: **Worst** = maximum steps over all inputs of size n (a guarantee). **Best** = minimum. **Average** = expected steps under a stated input distribution, e.g. linear search: $(n + 1)/2$ comparisons if the key is equally likely anywhere.

Amortized analysis: The **worst-case total** of a sequence of n operations divided by n — no probability. Dynamic array that doubles when full: n pushes cost $\leq n + (1 + 2 + 4 + \ldots + n) < 3n \Rightarrow$ **$O(1)$ amortized** per push, though one push costs $O(n)$. Methods: **aggregate**, **accounting** (banker's), **potential**.

Memory trick: "**Oh = ceiling, Omega = floor, Theta = both walls.**" Little o / ω are the strict versions — like < and >. To compare two functions, take lim f/g: $0 \to o$, constant $\to \Theta$, $\infty \to \omega$. Space complexity is analysed the same way, counting memory cells.

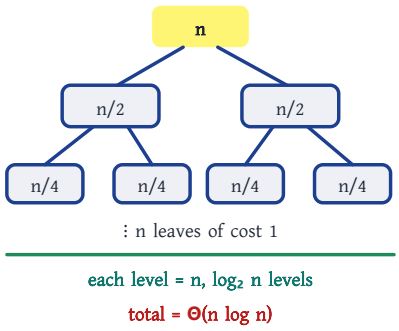
3 Recurrences & the Master Theorem

Definition: A **recurrence** gives T(n) in terms of T on smaller inputs. Solve it by **substitution** (guess, prove by induction), **iteration / recursion tree** (add the cost of every level), or the **Master theorem** for divide-and-conquer forms $T(n) = a \cdot T(n/b) + f(n)$, $a \geq 1$, $b > 1$.

Master theorem: Compare f(n) with **n^(log_b a)**, the total cost of the leaves.

1. **Case 1** — $f(n) = O(n^{(\log_b a - \varepsilon)})$ for some $\varepsilon > 0$ (leaves dominate) ⇒ **$T(n) = \Theta(n^{\log_b a})$**.
2. **Case 2** — $f(n) = \Theta(n^{\log_b a})$ (every level equal) ⇒ **$T(n) = \Theta(n^{\log_b a} \cdot \log n)$**. Extended: $f = \Theta(n^{\log_b a} \log^k n) \Rightarrow \Theta(n^{\log_b a} \log^{k+1} n)$.
3. **Case 3** — $f(n) = \Omega(n^{(\log_b a + \varepsilon)})$ **and** regularity $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$ (root dominates) ⇒ **$T(n) = \Theta(f(n))$**.

Short form for $f(n) = \Theta(n^d)$: $\log_b a > d \rightarrow \Theta(n^{\log_b a})$; $= d \rightarrow \Theta(n^d \log n)$; $< d \rightarrow \Theta(n^d)$.



recursion tree of $T(n) = 2T(n/2) + n$

Recurrence	leaves	Case	Solution
$T(n/2) + 1$	1	2	$\Theta(\log n)$
$2T(n/2) + 1$	n	1	$\Theta(n)$
$2T(n/2) + n$	n	2	**$\Theta(n \log n)$**
$4T(n/2) + n$	n^2	1	$\Theta(n^2)$
$4T(n/2) + n^2$	n^2	2	$\Theta(n^2 \log n)$
$4T(n/2) + n^3$	n^2	3	$\Theta(n^3)$
$7T(n/2) + n^2$	n^2.81	1	$\Theta(n^{\log_2 7})$
$3T(n/4) + n \log n$	n^0.79	3	$\Theta(n \log n)$

Not in Master form	Method	Solution
$T(n) = T(n-1) + 1 \cdot T(n-1) + n$	iteration: $1 + 1 + \ldots \cdot n + (n-1) + \ldots + 1$	$\Theta(n) \cdot$ **$\Theta(n^2)$**
$T(n) = 2T(n-1) + 1$	iteration (Hanoi)	$\Theta(2^n)$
$T(n) = T(n/3) + T(2n/3) + n$	recursion tree: $\leq n$ per level, $\log_{3/2} n$ levels	$\Theta(n \log n)$
$T(n) = 2T(n/2) + n \log n$	gap between cases → extended case 2	$\Theta(n \log^2 n)$

Worked example ✍️ substitution: Guess $T(n) \leq c \cdot n \log_2 n$ for $T(n) = 2T(n/2) + n$. Then $T(n) \leq 2 \cdot c(n/2)\log_2(n/2) + n = c \cdot n \log_2 n - c \cdot n + n \leq c \cdot n \log_2 n$ whenever **$c \geq 1$** ✓. Hence $T(n) = O(n \log n)$.

Memory trick: "**Leaves vs root.**" Work out n^(log_b a) first. Leaves bigger → case 1, answer = leaves. Tie → multiply by **log n**. Root bigger → answer = f(n).

Exam trap: The Master theorem does **not** apply to T(n-1) forms, $a < 1$, non-constant a (e.g. $2^n T(n/2)$), or a **non-polynomial gap** like n log n vs n. Case 3 also needs regularity.

4 Simple, Heap & Linear-Time Sorts

Vocabulary: **Stable**: equal keys keep their input order. **In-place**: O(1) extra memory (O(log n) stack is usually allowed). **Adaptive**: faster on nearly sorted input. **Online**: can sort as data arrives. An **inversion** is a pair $i < j$ with $a[i] > a[j]$; at most $n(n-1)/2$.

Sort	Idea	Key facts
Bubble	swap adjacent pairs; pass i floats the i-th largest to the end	$\le n(n-1)/2$ comparisons; swaps = inversions; **O(n) best** only with a "no swap → stop" flag
Selection	find the minimum of the rest, swap it to the front	always $n(n-1)/2$ comparisons, **$\le n-1$ swaps**; not stable, not adaptive
Insertion	insert a[i] into the sorted prefix by shifting	$n-1$ comparisons if sorted; shifts = inversions; stable, adaptive, online
Shell (1959)	insertion sort on gaps h, shrinking to 1	gaps n/2, n/4... → $O(n^2)$; Knuth 1, 4, 13, 40 → O(n^1.5); not stable
Heap	build max-heap O(n), then $n-1$ × (swap root, sift-down)	$\Theta(n \log n)$ always, O(1) space, not stable

Insertion	5 2 4 6 1 3
key 2	**2 5** 4 6 1 3
key 4	**2 4 5** 6 1 3
key 6	**2 4 5 6** 1 3
key 1	**1 2 4 5 6** 3
key 3	**1 2 3 4 5 6** ✓

Bubble	5 1 4 2 8
pass 1	1 4 2 5 **8**
pass 2	1 2 4 **5 8**
pass 3	no swap → **stop**
Selection	5^a 5^b 2 → 2 5^b 5^a
	5^a jumped past 5^b: unstable

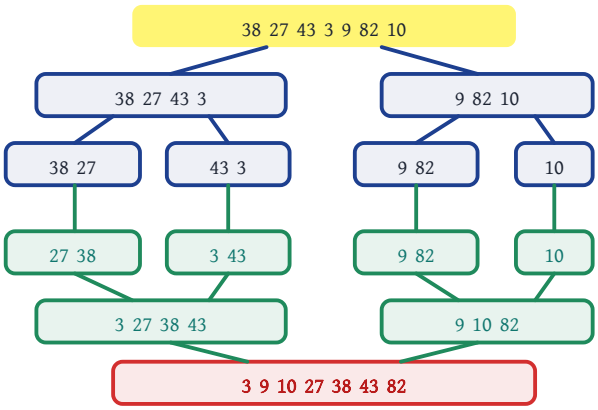
Linear-time (non-comparison) sorts: **Counting sort** (keys 0..k): count each key, prefix-sum the counts, place elements scanning **right to left** (keeps it stable) → $\Theta(n + k)$ time and space. A = 2 5 3 0 2 3 0 3: counts 2 0 2 3 0 1 → prefix 2 2 4 7 7 8 → 0 0 2 2 3 3 3 5. **Radix sort** (LSD): stable counting sort on each of d digits → **$\Theta(d(n + k))$**. **Bucket sort**: uniform input in [0, 1) into n buckets, insertion-sort each → $\Theta(n)$ average, $O(n^2)$ worst.

Worked example ✍ LSD radix sort: 170 45 75 90 802 24 2 66 → by units: 170 90 802 2 24 45 75 66 → by tens: 802 2 24 45 66 170 75 90 → by hundreds: **2 24 45 66 75 90 170 802**. Each pass must be **stable**, or the earlier digit order is lost.

Memory trick: "**Selection Swaps least, Insertion Shifts.**" Fewest writes → selection (n − 1 swaps). Nearly sorted or tiny → insertion (used inside Timsort and introsort).

Exam trap: Every comparison sort needs **Ω(n log n)** comparisons in the worst case ($\log_2 n!$ — proof in the NP-Completeness lesson). Counting and radix sort beat it only because they **never compare keys**.

5 Merge Sort, Quick Sort & the Sort Table



merge sort: blue = split (free), green = merge (the real work)

Merge sort: Split in half, sort both, **merge** in linear time: $T(n) = 2T(n/2) + \Theta(n) = \mathbf{\Theta(n \log n)}$ in **every** case. Merging lists of m and k items takes at most m + k − 1 comparisons (at least min(m, k)). Stable if ties take from the left (≤). Θ(n) extra array; ideal for **linked lists** and **external** (k-way) sorting.

Quick sort: (Hoare, 1961) Pick a **pivot**, **partition** so smaller keys sit left and larger right, recurse on both sides. Best/average Θ(n log n) (≈ 1.39 n $\log_2$ n comparisons); **worst $\mathbf{\Theta(n^2)}$** when the pivot is always the min or max, e.g. sorted input with first/last pivot: $T(n) = T(n-1) + \Theta(n)$. Fix: **random** or **median-of-three** pivot.

Lomuto, pivot 4	array (i = boundary)
start, i = −1	2 8 7 1 3 5 6 **4**
j=0: 2 ≤ 4, i=0	**2** 8 7 1 3 5 6 4
j=3: 1 ≤ 4, i=1	**2 1** 7 8 3 5 6 4
j=4: 3 ≤ 4, i=2	**2 1 3** 8 7 5 6 4
swap a[i+1], pivot	2 1 3 **4** 7 5 6 8

Memory trick: "**Merge: easy split, hard join. Quick: hard split, easy join.**" Both are divide-and-conquer (design method covered in the Design Methods lesson); merge works in **combine**, quick works in **divide**.

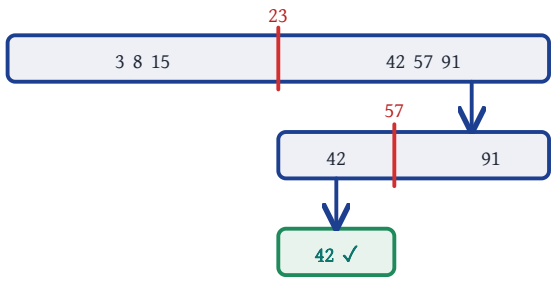
Algorithm	Best	Average	Worst	Extra space	Stable	In-place
Bubble (flag)	n	n^2	n^2	1	✓	✓
Selection	n^2	n^2	n^2	1	✗	✓
Insertion	n	n^2	n^2	1	✓	✓
Shell (Knuth)	n log n	n^1.25	n^1.5	1	✗	✓
Merge	n log n	n log n	n log n	n	✓	✗
Quick	n log n	n log n	**n^2**	log n	✗	✓
Heap	n log n	n log n	n log n	1	✗	✓
Counting	n + k	n + k	n + k	n + k	✓	✗
Radix (d digits)	d(n+k)	d(n+k)	d(n+k)	n + k	✓	✗
Bucket	n + k	n + k	n^2	n + k	✓	✗

Exam trap: (i) Quick sort's log n space is the **average** stack (naive worst O(n); recurse on the **smaller** side first). (ii) Selection sort is $\Theta(n^2)$ even on sorted input. (iii) "Fastest in practice" for RAM arrays → **quick sort**; "guaranteed n log n and stable" → **merge sort**; "guaranteed n log n in O(1) space" → **heap sort**.

6 Searching

Search	Needs	Best	Average	Worst
Linear	nothing; any list	O(1)	(n+1)/2	O(n)
Binary	**sorted + random access**	O(1)	O(log n)	**$\lceil \log_2(n+1) \rceil$**
Interpolation	sorted, uniform keys	O(1)	O(log log n)	O(n)
Ternary	sorted / unimodal	O(1)	$O(\log_3 n)$	O(log n)
Jump	sorted	O(1)	O(√n)	O(√n)
Hashing	a hash table	O(1)	O(1)	O(n)
Exponential	sorted, unbounded (i = position)	O(1)	O(log i)	O(log i)

```
int bsearch(const int a[], int n, int key) {
  int lo = 0, hi = n - 1;
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == key) return mid;
    if (a[mid] < key) lo = mid + 1;
    else              hi = mid - 1;
  }
  return -1;          /* not found */
}
```



n = 7 → at most 3 probes
7 → 3 → 1 elements left

search 42: mid 23 → go right, mid 57 → go left, mid 42 ✓

Worked example ✍️ trace and count: a = 3 8 15 23 42 57 91 (indices 0–6). **key 42**: (lo, hi, mid) = (0, 6, 3) → 23 < 42 → lo = 4; (4, 6, 5) → 57 > 42 → hi = 4; (4, 4, 4) → found at index 4 after **3** probes. **key 10**: mid 3 (23) → hi = 2; mid 1 (8) → lo = 2; mid 2 (15) → hi = 1; lo > hi → **-1**, also 3 probes. Maximum probes = $\lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil$: n = 7 → 3, **n = 1000 → 10**, n = 1024 → 11, $n = 10^6$ → 20.

The others in one line: **Interpolation**: pos = lo + (key - a[lo])·(hi - lo) / (a[hi] - a[lo]) — guesses like a dictionary. **Ternary**: two mids split into thirds; fewer iterations but more comparisons than binary, so binary wins on arrays — ternary is for the max/min of a unimodal function. **Jump**: step √n, then linear. **Exponential**: probe 1, 2, 4, 8 … then binary.

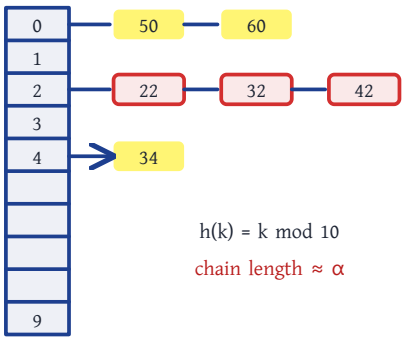
Exam trap: Binary search on a **linked list** is O(n) — no random access. On **unsorted** data it is simply wrong. mid = (lo + hi)/2 can **overflow** int; use lo + (hi - lo)/2. Linear search unsuccessful = n comparisons (n + 1 with a sentinel).

Memory trick: "**Halve until one: count the doublings.**" $2^{10} \approx 1000$, $2^{20} \approx 10^6$, $2^{30} \approx 10^9$ → 10, 20, 30 probes. Best case O(1): the key sits at the first mid. Used in C `bsearch`, Python `bisect`, B-tree node search, `git bisect`.

7 Hashing & Collision Resolution

Definition: **Hashing** maps a key k to a slot h(k) ∈ {0 … m - 1} of a table of size m, giving search/insert/delete in **O(1) average** (O(n) worst). Two keys with the same slot **collide**. **Load factor α = n / m** (keys ÷ slots) decides the speed.

Function	Rule	Example (m = 100 unless stated)
Division	k mod m; m prime, not near 2^p	m = 11: 59 → 4
Multiplication	⌊m · frac(kA)⌋, A ≈ 0.618 (Knuth)	123A = 76.018 → 1
Mid-square	square k, take the middle digits	56^2 = 3136 → 13
Folding	split into parts and add, drop carry	123+456+789 = 1368 → 368 (m = 1000)



separate chaining, m = 10: each slot heads a linked list

Scheme	Probe i (all mod m), α ≤ 1
Linear	h + i → **primary clustering**
Quadratic	$h + c_1 i + c_2 i^2$ (often $h + i^2$) → secondary clustering
Double	$h_1 + i \cdot h_2$; $h_2 \neq 0$, coprime to m → near uniform
Chaining	a list per slot; α may exceed 1

Worked example ✍️ insert 10, 22, 31, 4, 15, 28, 17, 88, 59 into m = 11, h_1 = k mod 11:

slot	0	1	2	3	4	5	6	7	8	9	10
linear = quadratic	22	88	–	–	4	15	28	17	**59**	31	10
double	22	–	**59**	17	4	15	28	88	–	31	10

59 → h_1 = 4. Linear: 4, 5, 6, 7 full → slot 8 (**5 probes**). Quadratic: 4, 5 → 4 + 4 = 8 (3 probes). Double (h_2 = 1 + k mod 10): $h_2(59)$ = 10, so 4 → 14 mod 11 = 3 (17) → 24 mod 11 = **2**.

Expected probes	Unsuccessful	Successful	α = 0.5
Chaining	1 + α	1 + α/2	1.5 · 1.25
Uniform open addressing	**1/(1 − α)**	(1/α) ln 1/(1 − α)	2 · 1.39
Linear probing (Knuth)	½(1 + 1/(1 − α)^2)	½(1 + 1/(1 − α))	2.5 · 1.5

Exam trap: Open addressing cannot simply empty a slot on delete — later keys become unreachable; mark it **DELETED** (tombstone). Quadratic probing may miss free slots unless m is prime and α ≤ ½. Rehash (double m) when α passes a threshold (0.75 in Java's HashMap).

Memory trick: "**Linear Lumps, Quadratic Quietens, Double Disperses.**" Probe count blows up near α = 1: α = 0.9 → 10 probes for a miss. Uses: symbol tables, DB hash indexes, caches (Redis, Memcached), sets/dictionaries.

8 Storage Management & Garbage Collection

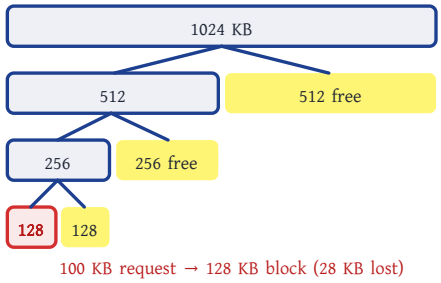
Definition: **Storage management** hands out and takes back **heap** blocks (malloc/free, new/delete). Free blocks sit on a **free list**; a **placement policy** serves each request and freed neighbours are **coalesced**. The enemies: **fragmentation** and **garbage** (unreachable, unreleased blocks).

Placement policies: **First fit**: first hole big enough — fastest. **Next fit**: first fit resumed where the last search stopped. **Best fit**: the **smallest** hole that fits — scans the whole list, leaves tiny slivers. **Worst fit**: the largest hole — leaves big leftovers. In the example below **only best fit** places all four.

Worked example ✍️ holes 100, 500, 200, 300, 600 KB; requests 212, 417, 112, 426:

Policy	212	417	112	426
First fit	500 (→288)	600 (→183)	288 (→176)	waits
Best fit	300 (→88)	500 (→83)	200 (→88)	**600 ✓**
Worst fit	600 (→388)	500 (→83)	388 (→276)	waits

Fragmentation: **External**: enough free memory in total but no single hole is large enough — cured by **compaction** (slide live blocks together; needs run-time relocation, costly). **Internal**: a block larger than the request — the unused tail is wasted (buddy system, fixed partitions, paging).



buddy split of 1 MB

Buddy system: Blocks are powers of 2. A request is **rounded up** to 2^k; a larger block is split into two halves (**buddies**) until it fits. On free, a block merges with its buddy if the buddy is free; buddy address = addr **XOR** size. Fast split/merge, but internal fragmentation up to ~50%.

Garbage collector	How it works	Strength · weakness
Reference counting	free when pointer count = 0	immediate · **fails on cycles**
Mark-and-sweep	mark from **roots** (stack, globals), sweep rest	handles cycles · pauses (Lisp, 1960)
Copying	copy live objects to the other semi-space	compacts · wastes half the heap
Generational	collect the young area often	short pauses (Java, .NET)

Exam trap: Two unreachable objects pointing at each other keep count 1 → **leak** under pure reference counting. **Dangling pointer** = freed but still used; **leak** = unused but never freed. Compaction cures external fragmentation only.

Memory trick: "**First is Fast, Best leaves Bits, Worst leaves Wide.**" GC: "**count, mark, copy**" — counting misses cycles, copying needs twice the room.

Night-Before-the-Exam Sheet

Fact sheet:

Hanoi: $2^n - 1$ moves

naive fib: $\Theta(\varphi^n)$, $2F(n+1) - 1$ calls

recursion depth d → O(d) stack

tail call → reusable frame

$O \le \cdot\ \Omega \ge \cdot\ \Theta = \cdot\ o < \cdot\ \omega >$

$\log n! = \Theta(n \log n)$

Master: compare f with n^(log_b a)

tie ⇒ × log n

$2T(n/2) + n = \mathbf{\Theta(n \log n)}$

$T(n/2) + 1 = \Theta(\log n)$

$T(n-1) + n = \Theta(n^2)$

$7T(n/2) + n^2 = \Theta(n^{\wedge}2.81)$

comparison sort $\ge \mathbf{\Omega(n \log n)}$

counting $\Theta(n + k)$, radix $\Theta(d(n + k))$

binary search $\le \mathbf{\lceil \log_2(n+1) \rceil}$

interpolation O(log log n) avg

$\alpha = n/m$ · miss $\approx 1/(1 - \alpha)$

linear probe → primary cluster

buddy: round up to 2^k

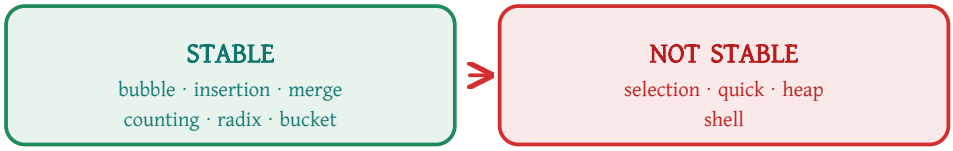
ref-count misses cycles

Top 12 one-liners for MCQs

1. Stable: **bubble, insertion, merge**, counting, radix
2. Not stable: **selection, quick, heap**, shell
3. Quick worst $\Theta(n^2)$: sorted input, end pivot
4. Merge: $\Theta(n \log n)$ always, $\Theta(n)$ space
5. Heap: $\Theta(n \log n)$ always, $O(1)$ space
6. Fewest swaps: **selection** $(n - 1)$
7. Best for nearly sorted: **insertion**
8. Binary search needs sorted + random access
9. Max probes, n = 1000 → **10**
10. Double hashing ≈ uniform hashing
11. Delete in open addressing → tombstone
12. Only best fit placed all four requests

Top traps: O is a bound, not "the worst case" · Master theorem needs $a \cdot T(n/b)$, not $T(n-1)$ · bubble sort is $O(n)$ best **only** with the early-exit flag · quick sort's $O(\log n)$ space is average stack · binary search on a linked list is $O(n)$ · mid = (lo + hi)/2 can overflow · emptying a slot breaks open-addressing search · amortized ≠ average case.

All tricks together: "base first, smaller next" · "Oh = ceiling, Omega = floor, Theta = both walls" · "leaves vs root" · "Selection Swaps least, Insertion Shifts" · "Merge: easy split, hard join; Quick: hard split, easy join" · "halve until one — 10, 20, 30" · "Linear Lumps, Quadratic Quietens, Double Disperses" · "First is Fast, Best leaves Bits, Worst leaves Wide" · "count, mark, copy".



the stability split every MCQ paper tests

✓ Practice MCQs (exam pattern, 10 marks)

1. Moves for Hanoi with 5 discs:
   (a) 31 (b) 32
   (c) 25 (d) 63
2. T(n) = 2T(n/2) + n is:
   (a) $\Theta(n)$ (b) $\Theta(n^2)$
   (c) $\Theta(n \log n)$ (d) $\Theta(\log n)$
3. Stable in standard form:
   (a) quick sort (b) heap sort
   (c) selection (d) merge sort
4. Quick sort, first pivot, sorted input:
   (a) $O(n \log n)$ (b) $O(n^2)$
   (c) $O(n)$ (d) $O(\log n)$
5. Max binary-search probes, n = 1000:
   (a) 9 (b) 10
   (c) 11 (d) 500
6. m = 7, k mod 7, linear probing; 10, 17, 24 inserted. 24 goes to:
   (a) 3 (b) 4
   (c) 5 (d) 6
7. f = $\Theta(g)$ means g is:
   (a) upper only (b) lower only
   (c) average case (d) tight bound
8. Cannot reclaim cyclic garbage:
   (a) mark-sweep (b) ref counting
   (c) copying (d) generational

9. $T(n) = 4T(n/2) + n^2$ is:

(a) $\Theta(n^2)$ (b) $\Theta(n^2 \log n)$

(c) $\Theta(n^3)$ (d) $\Theta(n \log n)$

10. Buddy system, 1 MB pool, 100 KB request gets:

(a) 100 KB (b) 64 KB

(c) 128 KB (d) 256 KB

Answer key → 1-a · 2-c · 3-d · 4-b · 5-b · 6-c · 7-d · 8-b · 9-b · 10-c

Written question: State the Master theorem and solve $T(n) = 2T(n/2) + n$ and $T(n) = 7T(n/2) + n^2$. Trace Lomuto partition on 2 8 7 1 3 5 6 4 (pivot = last). Give quick sort's best and worst cases with their recurrences, and one way to avoid the worst case. 10

1. **Theorem** — $T(n) = aT(n/b) + f(n)$, $a \geq 1$, $b > 1$. Compare f with $n^{\log_b a}$: smaller by $n^{\varepsilon} \rightarrow \Theta(n^{\log_b a})$; equal → $\Theta(n^{\log_b a} \log n)$; larger by n^{ε} with $af(n/b) \leq cf(n)$, $c < 1$ → $\Theta(f(n))$. 3
2. **Solve** — $2T(n/2) + n$: $n^{(\log_2 2)} = n = f$ → case 2 → **$\Theta(n \log n)$**. $7T(n/2) + n^2$: $n^{(\log_2 7)} \approx n^{2.81} > n^2$ → case 1 → **$\Theta(n^{\log_2 7})$**. 2
3. **Partition** — $i = -1$; $j = 0$ (2 ≤ 4) $i = 0$; $j = 3$ (1) $i = 1$ → 2 1 7 8 3 5 6 4; $j = 4$ (3) $i = 2$ → 2 1 3 8 7 5 6 4; swap a[3] with pivot → **2 1 3 4 7 5 6 8**, pivot at index 3. 3
4. **Cases** — best: balanced split, $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$. Worst: split 0 / $n - 1$, $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$ (sorted input, end pivot). Avoid with a random or median-of-three pivot. 2

Before the exam ✓

- ✓ Can solve any $aT(n/b) + f(n)$ in 30 s
- ✓ Can fill the sort table from memory
- ✓ Can insert keys with 3 probing schemes
- ✓ Can count binary-search probes